

Equazione del Calore 2D e Riordinamento nelle Fattorizzazioni Cholesky

Documento di Progetto
Informatica con Laboratorio – A.A. 2025/2026

Giosuè Aiello

1 Impostazione del problema

L'obiettivo del progetto è la risoluzione numerica dell'equazione del calore in regime stazionario in due dimensioni spaziali. Il dominio di calcolo è il quadrato unitario $\Omega = [0, 1]^2$, e si cerca la funzione di temperatura $u : \Omega \rightarrow \mathbb{R}$ che soddisfa l'equazione ellittica:

$$\kappa \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + f(x, y) = 0, \quad (x, y) \in \Omega, \quad (1)$$

con condizioni al contorno di Dirichlet sul bordo $\partial\Omega$:

$$u(x, y) = u_0(x, y), \quad (x, y) \in \partial\Omega.$$

I parametri fisici del problema sono: coefficiente di diffusività termica $\kappa = 0.01$ e termine sorgente $f(x, y) = e^{-10(x^2+y^2)}$, che modella una sorgente di calore concentrata in prossimità dell'origine con decadimento gaussiano. Nel caso base si considera $u_0 \equiv 0$ (condizioni omogenee).

Discretizzazione alle differenze finite

Si introduce una griglia cartesiana uniforme di $(N + 2)^2$ punti (x_i, y_j) con $i, j = 0, \dots, N + 1$ e passo $h = 1/(N + 1)$, dove $x_i = i \cdot h$ e $y_j = j \cdot h$. I punti di bordo hanno temperatura fissata e non contribuiscono come incognite. Le N^2 incognite del problema sono i valori $u_{i,j} \approx u(x_i, y_j)$ nei punti *interni* ($1 \leq i, j \leq N$).

Approssimando le derivate seconde con lo stencil a cinque punti, l'equazione (1) valutata nel punto interno (x_i, y_j) diventa:

$$\frac{\kappa}{h^2} (u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}) + f(x_i, y_j) = 0. \quad (2)$$

L'insieme delle equazioni (2) costituisce il sistema lineare $Au = b$, dove $A \in \mathbb{R}^{N^2 \times N^2}$ è sparsa (al più 5 elementi non nulli per riga), simmetrica e definita negativa.

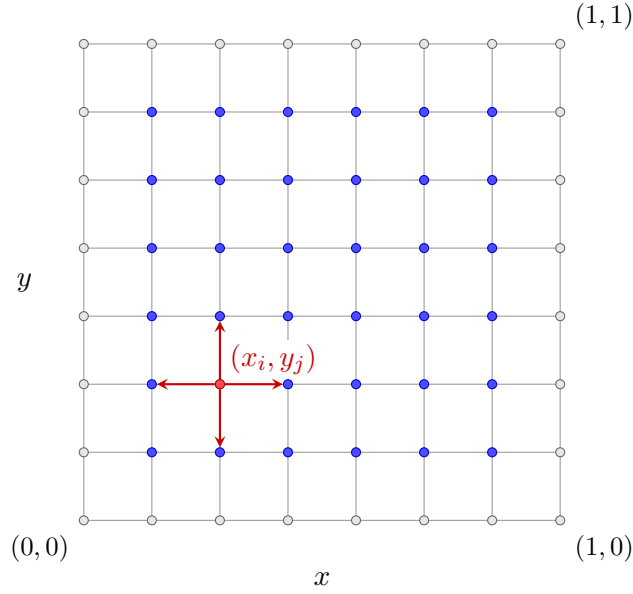


Figura 1: Griglia di discretizzazione: nodi di bordo (grigio), nodi interni (blu) e i quattro vicini dello stencil per il nodo (x_i, y_j) (freccie rosse).

Fattorizzazione di Cholesky e fill-in

Poiché $-A$ è simmetrica definita positiva (SPD), si calcola la fattorizzazione di Cholesky $-A = LL^T$ con L triangolare inferiore sparsa. Durante la fattorizzazione si generano elementi non nulli in posizioni che erano zero in $-A$: questo fenomeno è detto *fill-in* e dipende fortemente dall'ordine con cui le incognite sono enumerate.

Con l'ordinamento naturale per righe la larghezza di banda di A è $\Theta(N)$ e il fill-in di L è $O(N^3)$. La tecnica di *nested dissection* riduce il fill-in a $O(N^2 \log N)$, con un risparmio sostanziale per valori grandi di N .

Nested dissection: approccio geometrico

La struttura di A è codificata dal *grafo di adiacenza* $G = (V, E)$: ogni nodo $v \in V$ corrisponde a una variabile $u_{i,j}$ e ogni arco $(u, v) \in E$ indica che $A_{uv} \neq 0$. Per la griglia regolare:

$$(x_i, y_j) \leftrightarrow (x_k, y_l) \iff (i = k, |j - l| = 1) \text{ oppure } (j = l, |i - k| = 1).$$

La *nested dissection* individua ricorsivamente un separatore V_S che divide V in due insiemi bilanciati V_1 e V_2 senza archi diretti tra loro. L'ordinamento risultante elenca prima V_1 , poi V_2 , infine V_S , in modo che il separatore appaia in fondo alla matrice, limitando la propagazione del fill-in.

In questo progetto si adotta un **approccio geometrico puro**: il separatore non è determinato esplorando il grafo di adiacenza, bensì è definito direttamente tramite le coordinate spaziali dei nodi. Scelto l'asse di taglio $c \in \{x, y\}$ e calcolato l'indice mediano $\hat{c}$, si pone:

$$V_1 = \{u_{i,j} \mid c < \hat{c}\}, \quad V_2 = \{u_{i,j} \mid c > \hat{c}\}, \quad V_S = \{u_{i,j} \mid c = \hat{c}\}.$$

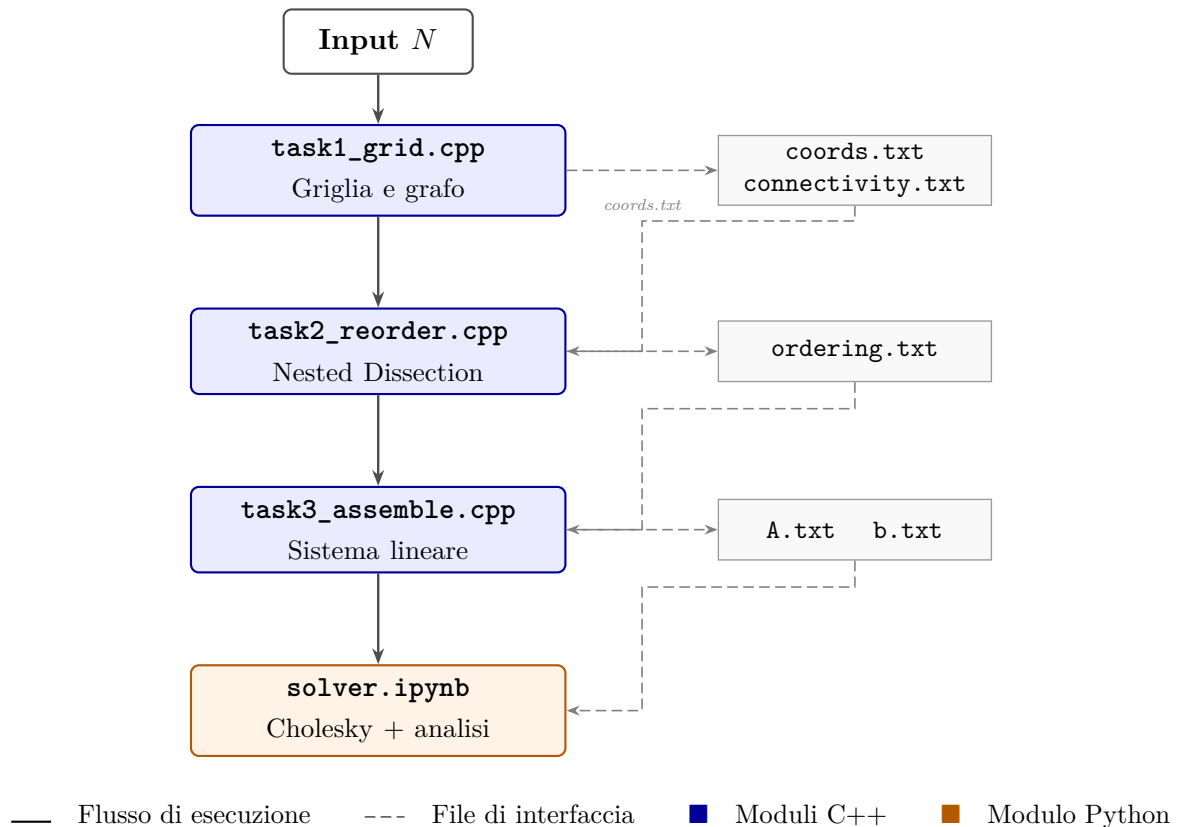
Il procedimento viene applicato ricorsivamente a V_1 e V_2 , alternando il taglio lungo x e y . Per griglie cartesiane regolari, il piano mediano è *al contempo* un separatore geometrico e un separatore topologico del grafo: non esistono archi di adiacenza tra V_1 e V_2 (i punti sono separati

da almeno un passo reticolare lungo l'asse di taglio), quindi l'approccio geometrico produce esattamente il medesimo separatore ottimale che si otterrebbe esplorando la topologia del grafo, senza il costo di tale esplorazione.

2 Architettura del progetto

Modulo principale: HES-HeatEquationSolver

Il sistema viene concepito come un unico risolutore logico denominato HES-HeatEquationSolver, articolato in quattro sotto-moduli specializzati. I tre moduli in C++ comunicano tra loro attraverso file di testo con formato fisso che fungono da interfaccia verificabile indipendentemente; il modulo Python costituisce il punto di ingresso per la fase numerica finale.



Le strutture dati fondamentali dell'intero sistema sono:

- **Liste di adiacenza** (`std::vector<std::vector<int>>`) nel Modulo 1 per la costruzione e scrittura del grafo di adiacenza.
- **Vettore di nodi geometrici** (`std::vector<Node>`) nel Modulo 2: `struct Node` contiene `id`, `i`, `j`, `x`, `y`; consente la nested dissection geometrica senza caricare in memoria la lista di adiacenza.
- **Ricorsione classica** nel Modulo 2: l'algoritmo divide-et-impera è implementato ricorsivamente come da specifiche, potendo contare su una profondità limitata dell'albero delle chiamate.
- **Vettori di permutazione** (`std::vector<int> perm`, `inv_perm`) per la mappatura bidirezionale tra ordinamento naturale e riordinato.
- **Formato sparso CSC** (`scipy.sparse.csc_matrix`) per la matrice A nel modulo Python, richiesto nativamente da CHOLMOD.

3 Specifiche dei moduli

I quattro sotto-moduli vengono descritti con livello di dettaglio crescente. Per ciascuno si indicano input, output, funzionalità richieste, strutture dati, invarianti, dipendenze e complessità attesa.

Modulo 1 – `task1_grid.cpp`: generazione della griglia e del grafo

Il Modulo 1 è responsabile della creazione della struttura geometrica del dominio e del relativo grafo di adiacenza. Costituisce il punto di ingresso dell'intera pipeline e non dipende da nessun altro modulo.

Formalizzazione matematica

Il dominio $\Omega = [0, 1]^2$ viene discretizzato con passo $h = \frac{1}{N+1}$. L'insieme dei nodi interni (le incognite) è:

$$V = \{(x_i, y_j) \mid x_i = i \cdot h, y_j = j \cdot h, \quad i, j \in \{1, \dots, N\}\}, \quad |V| = N^2.$$

Il grafo di adiacenza $G = (V, E)$ ha archi:

$$E = \left\{ \{u, v\} \subset V \mid \sqrt{(x_i - x_k)^2 + (y_j - y_l)^2} = h \right\}.$$

Ogni nodo interno ha al massimo 4 vicini (sinistra, destra, basso, alto). A ogni nodo viene assegnato un identificatore intero univoco tramite la mappa $\phi(i, j) = (i - 1) \cdot N + (j - 1)$.

Specifiche

- **Input:** Intero $N > 0$ passato da riga di comando; se assente o non valido, si attiva un prompt iterativo con opzione di uscita (q).
- **Output:**
 - `output/coords.txt` – una riga per nodo: `id i j x y`
 - `output/connectivity.txt` – una riga per arco: `e u v` (dove `e` è l'indice progressivo dell'arco e $u < v$)
- **Funzionalità richieste:**
 - Parsing e validazione di N con fallback interattivo a ciclo (loop).
 - Generazione dei N^2 nodi con ID progressivo e coordinate spaziali (precisione `double`).
 - Costruzione della lista di adiacenza bidirezionale e scrittura degli archi senza duplicati ($u < v$).
 - Creazione della directory `output/` se non esiste.
- **Strutture dati:** `std::vector<std::vector<int>>` di dimensione N^2 , pre-allocata con `reserve(4)` per ciascun nodo.
- **Invarianti:**
 - La lista di adiacenza è sempre simmetrica: se $v \in \text{adj}[u]$ allora $u \in \text{adj}[v]$.
 - Ogni arco appare esattamente una volta in `connectivity.txt` (condizione $u < v$).
 - Il numero di righe in `coords.txt` è esattamente N^2 .

- **Dipendenze:** Nessuna. Questo modulo è il primo della pipeline.
- **Complessità attesa:** $\mathcal{O}(N^2)$ in tempo e spazio.

Logica implementativa (pseudo-codice)

```

1 Leggi N da argv; se non valido chiedi in loop finche' N>0 o utente esce
2 Crea cartella output/ se non esiste
3 Per i = 1 .. N:
4   Per j = 1 .. N:
5     id = (i-1)*N + (j-1)
6     x  = i/(N+1), y = j/(N+1)
7     Scrivi (id, i, j, x, y) su coords.txt
8     Se j < N: aggiungi arco bidirezionale (id, id+1)
9     Se i < N: aggiungi arco bidirezionale (id, id+N)
10 Per ogni nodo u, per ogni vicino v:
11   Se u < v: scrivi (e++, u, v) su connectivity.txt

```

Modulo 2 – task2_reorder.cpp: nested dissection geometrica

Il Modulo 2 costituisce il cuore algoritmico del progetto. Calcola una permutazione ottimizzata dei nodi tramite la *nested dissection geometrica*, al fine di minimizzare il fill-in durante la fattorizzazione di Cholesky.

Approccio geometrico vs. approccio su grafo

Esistono due famiglie di algoritmi per la nested dissection:

1. **Approccio su grafo** (es. bisection spettrale, METIS): esplora esplicitamente il grafo di adiacenza $G = (V, E)$, calcola autovettori del Laplaciano o partiziona tramite BFS/Kernighan-Lin. Richiede $\mathcal{O}(|E|) = \mathcal{O}(N^2)$ solo per costruire e visitare il grafo, più il costo delle euristiche di partizionamento (generalmente $\mathcal{O}(N^2\alpha(N))$ con α lenta ma non banale). Tali metodi sono necessari per griglie non strutturate o domini irregolari.
2. **Approccio geometrico** (adottato in questo progetto): ignora completamente la lista di adiacenza. Il separatore viene individuato tramite il *piano mediano* sull'asse corrente: dati gli indici interi (i, j) dei nodi, si calcola $\hat{c} = \lfloor (\min_c + \max_c)/2 \rfloor$ e si partiziona $V_S = \{v \mid c_v = \hat{c}\}$, $V_1 = \{v \mid c_v < \hat{c}\}$, $V_2 = \{v \mid c_v > \hat{c}\}$.

Perché l'approccio geometrico è superiore per griglie regolari. Per una griglia cartesiana uniforme $N \times N$ l'approccio geometrico è *equivalente* a quello su grafo in termini di qualità del separatore, ma *superiore* sotto ogni aspetto implementativo:

- **Correttezza del separatore:** il piano mediano $c = \hat{c}$ divide la griglia in due metà con al massimo $\lceil N/2 \rceil$ nodi ciascuna. Non esistono archi di adiacenza tra V_1 e V_2 (i nodi adiacenti si trovano a distanza reticolare h lungo l'asse di taglio, quindi il nodo con $c = \hat{c}$ funge da buffer). Il separatore geometrico *coincide* esattamente con il separatore topologico del grafo.
- **Complessità:** $\mathcal{O}(N^2 \log N)$ deterministico, senza euristiche. L'approccio su grafo richiede almeno $\mathcal{O}(N^2)$ per la sola lettura degli archi, più il costo dell'algoritmo di partizionamento.

- **Efficienza in cache:** accesso sequenziale al solo vettore di nodi (`coords.txt` $\rightarrow$ `std::vector<Node>`), nessuna lista di puntatori per adiacenze.
- **Semplicità e verificabilità:** l'invariante “ V_S è il piano mediano” è dimostrabile analiticamente; l'invariante di un separatore trovato per BFS su grafo non strutturato richiede verifica sperimentale.
- **Robustezza:** utilizzo di confronti su indici interi $((i, j))$ anziché coordinate floating-point, eliminando i rischi di arrotondamento.

Formalizzazione matematica

L'algoritmo riceve in input il vettore S di nodi con coordinate intere (i, j) e reali (x, y) , e individua ricorsivamente:

$$V_1 = \{u \in S \mid c_u < \hat{c}\}$$

$$V_2 = \{u \in S \mid c_u > \hat{c}\}$$

$$V_S = \{u \in S \mid c_u = \hat{c}\}$$

dove $c \in \{i, j\}$ è l'indice dell'asse di taglio e $\hat{c} = \lfloor (\min_{u \in S} c_u + \max_{u \in S} c_u) / 2 \rfloor$. L'ordinamento risultante è $V_1 \rightarrow V_2 \rightarrow V_S$, ricorsivo con alternanza degli assi.

Specifiche

- **Input:** `output/coords.txt` (prodotto dal Modulo 1). Il file `connectivity.txt` *non viene letto*: per l'approccio geometrico la struttura topologica del grafo è ridondante.
- **Output:** `output/ordering.txt` – una riga per posizione: `new_id old_id`
- **Funzionalità richieste:**
 - Lettura delle sole coordinate da `coords.txt`.
 - Partizionamento tramite ricorsione classica (funzione che richiama se stessa).
 - Calcolo del piano mediano con indici interi, con alternanza degli assi i/j ad ogni livello.
 - Verifica dell'invariante: `ordering.size() == N2`.
 - Salvataggio dell'ordinamento su `ordering.txt`.
- **Strutture dati:** `struct Node {id, i, j, x, y}; std::vector<int> ordering`.
- **Invarianti:**
 - Il vettore `ordering` è una permutazione valida di $\{0, \dots, N^2 - 1\}$.
 - I nodi di V_S compaiono sempre *dopo* i nodi di V_1 e V_2 alla stessa profondità.
 - L'asse di taglio si alterna ad ogni livello ($0 = i, 1 = j$).
- **Dipendenze:** Modulo 1 (solo `coords.txt`).
- **Complessità attesa:** $\mathcal{O}(N^2 \log N)$ in tempo; $\mathcal{O}(N^2)$ in spazio.

Logica implementativa (pseudo-codice)

```

1  Carica nodi da output/coords.txt -> vector<Node>
2  Funzione ricorsiva ND(S, asse):
3      Se |S| == 0: ritorna
4      Se |S| == 1: aggiungi S[0].id a ordering; ritorna
5      Calcola min_val, max_val su asse corrente (indice i o j)
6      mid_val = (min_val + max_val) / 2
7      Partiziona S in V1 (val<mid), V2 (val>mid), VS (val==mid)
8      next_axis = 1 - asse
9      Richiama ND(V1, next_axis)
10     Richiama ND(V2, next_axis)
11     Per ogni nodo in VS: aggiungi a ordering
12 (Nel main):
13     Chiama ND(tutti_i_nodi, 0)
14     Verifica: ordering.size() == nodes.size()
15     Scrivi output/ordering.txt: per k=0..N^2-1, scrivi "k ordering[k]"

```

Modulo 3 – task3_assemble.cpp: assemblaggio del sistema lineare

Il Modulo 3 costruisce il sistema lineare $Au = b$ che approssima l'equazione differenziale. Supporta sia l'ordinamento naturale che quello ottimizzato via il flag `-reorder`, e implementa la parte opzionale delle condizioni al bordo non omogenee.

Formalizzazione matematica

Applicando lo stencil a 5 punti con passo $h = 1/(N + 1)$ e diffusività κ , i coefficienti della matrice per ogni nodo k (nel nuovo ordinamento) sono:

$$A_{k,k} = -\frac{4\kappa}{h^2}$$

$$A_{k,k'} = +\frac{\kappa}{h^2} \quad \text{per ogni vicino interno } k'$$

Il termine noto è $b_k = -f(x_{i(k)}, y_{j(k)})$. Se il vicino k' cade sul bordo (condizione di Dirichlet), il suo valore u_0 è noto e viene incorporato nel termine noto:

$$b_k \leftarrow \frac{\kappa}{h^2} \cdot u_0(x_{\text{bordo}}, y_{\text{bordo}}).$$

Specifiche

- **Input:** Intero N e flag opzionale `-reorder` da riga di comando; `output/coords.txt`; `output/ordering.txt` (solo se `-reorder`).
- **Output:** `output/A.txt` (triplette $i \ j \ val$) e `output/b.txt` (una riga per valore di b).
- **Funzionalità richieste:**
 - Lettura di `coords.txt` e costruzione della mappa $\phi^{-1} : (i, j) \rightarrow \text{id}$.
 - Se `-reorder`: lettura di `ordering.txt` e costruzione di `perm` e `inv_perm`; altrimenti vettori identità.
 - Menu interattivo per la scelta della condizione al bordo u_0 (selezione avviene una sola volta, prima del ciclo di assemblaggio).

- Assemblaggio dello stencil con gestione delle condizioni di Dirichlet.
- Scrittura di `A.txt` e `b.txt` con precisione floating-point a 10 cifre decimali.
- **Strutture dati:** `std::vector<int> perm, inv_perm` di dimensione N^2 ; `struct Triplet {int row, col; double val}; std::vector<Triplet>` pre-allocato con `reserve(5*N*N)`; `std::vector<double> b`.
- **Invarianti:**
 - `perm` e `inv_perm` sono permutazioni inverse l'una dell'altra: `inv_perm[perm[k]] == k` per ogni k .
 - La matrice A è simmetrica: se la tripletta (k, k', v) è presente, lo è anche (k', k, v) .
 - Il numero di righe in `b.txt` è esattamente N^2 .
- **Dipendenze:** Modulo 1 (`coords.txt`); Modulo 2 (`ordering.txt`, solo se `-reorder`).
- **Complessità attesa:** $\mathcal{O}(N^2)$ in tempo (al più 5 entrate per nodo) e spazio.

Logica implementativa (pseudo-codice)

```

1 Leggi parametri argv: N, --reorder (opz), e scelta u0 (0..5)
2 Carica coords.txt in mappa coords[id] = {i, j, x, y}
3 Se --reorder: carica ordering.txt, costruisci perm e inv_perm
4             altrimenti: perm[k] = inv_perm[k] = k
5 Per k = 0 .. N^2 - 1:
6     old_id = perm[k]; leggi (i, j, x, y) da coords[old_id]
7     Aggiungi triplet (k, k, -4*kappa/h^2)
8     b[k] = -f(x, y)
9     Per ogni direzione d in {su, giu, sx, dx}:
10        (ni, nj) = vicino di (i,j) in direzione d
11        Se (ni,nj) fuori dominio:
12            b[k] -= (kappa/h^2) * u0(ni*h, nj*h)
13        Altrimenti:
14            k_vic = inv_perm[id_naturale(ni,nj)]
15            Aggiungi triplet (k, k_vic, +kappa/h^2)
16 Scrivi A.txt (triplet) e b.txt (vettore b)

```

Condizioni al bordo disponibili (parte opzionale)

Scelta	Funzione $u_0(x, y)$	Utilità di verifica
0	0	Caso base omogeneo (default)
1	10	Con $f = 0$, la soluzione deve essere $\equiv 10$
2	x	Con $f = 0$, la soluzione interna deve coincidere con x
3	$\sin(\pi x)$	Bordo continuo; si annulla agli angoli
4	$x^2 - y^2$	$\nabla^2(x^2 - y^2) = 0$: soluzione numerica esatta
5	$\mathbf{1}_{y=1}$	Shock termico: solo il bordo superiore è caldo

Modulo 4 – `solver.ipynb`: risoluzione (Task 4) e analisi (Task 5)

Il Modulo 4 è il punto terminale della pipeline. In termini di specifiche progettuali, questo modulo accorpa coerentemente sia le richieste della **Task 4** (costruzione della matrice sparsa CSC e fattorizzazione di Cholesky) sia della **Task 5** (benchmark per $N \in \{32, \dots, 1024\}$, *spy plot* della struttura e grafici della soluzione).

Si è optato per questa scelta architetturale poiché entrambe le task si collocano naturalmente nello stesso ambiente di esecuzione (Python). Piuttosto che frammentare la logica in script separati, un unico Jupyter Notebook (`solver.ipynb`) funge da orchestratore finale: legge i dati generati dal Modulo 3, esegue i calcoli numerici e restituisce contestualmente l'analisi visiva e prestazionale.

Specifiche

- **Input:**
 - `output/A.txt` e `output/b.txt` (prodotti dal Modulo 3).
 - Parametro N (o serie $N \in \{32, 64, 128, 256, 512, 1024\}$ per il benchmark).
- **Output:**
 - Vettore soluzione $u \in \mathbb{R}^{N^2}$, rimappato sulla griglia originale per la visualizzazione.
 - Spy plot della matrice A e del fattore L per ordinamento naturale e nested dissection.
 - Mappa di calore 2D e superficie 3D della soluzione $u(x, y)$.
 - Grafico log-log del fill-in `nnz(L)` e dei tempi di fattorizzazione al variare di N .
- **Funzionalità richieste:**
 - Lettura di `A.txt` e costruzione di `scipy.sparse.coo_matrix`, con conversione in `csc_matrix`.
 - Fattorizzazione $-A = LL^T$ tramite `sksparse.cholmod.cholesky` con `order="natural"`.
 - Risoluzione del sistema con `scipy.sparse.linalg.spsolve_triangular` (forward e backward substitution).
 - Automazione dell'intera pipeline C++ tramite `subprocess` per ciascun valore di N e ordinamento.
 - Misurazione dei tempi con `time.perf_counter`.
- **Strutture dati:**
 - `scipy.sparse.coo_matrix` per l'assemblaggio iniziale; `csc_matrix` per la fattorizzazione.
 - `numpy.ndarray` per la soluzione u e il termine noto b .
 - Oggetto `Factor` di `CHOLMOD` per estrarre la matrice L esplicita.
- **Invarianti:**
 - La matrice costruita da `coo_matrix` deve essere simmetrica e definita negativa.
 - Il vettore soluzione u deve soddisfare $\|Au - b\|_2 / \|b\|_2 < 10^{-10}$ (residuo relativo). Questa severa soglia assicura l'assenza di cancellazioni catastrofiche prodotte dall'accumulo

di errori in virgola mobile, tollerando al contempo il fisiologico malcondizionamento $\mathcal{O}(N^2)$ della matrice Laplaciana all'aumentare di N .

- **Dipendenze:** Modulo 3 (A.txt, b.txt); librerie `scipy`, `numpy`, `scikit-sparse`, `matplotlib`.
- **Complessità attesa:** Fill-in $\mathcal{O}(N^3)$ con ordinamento naturale; $\mathcal{O}(N^2 \log N)$ con nested dissection. Tempo di fattorizzazione dominato dal fill-in.

Logica implementativa (pseudo-codice)

```

1 Per N in {32, 64, 128, 256, 512, 1024}:
2   Per ordering in {naturale, nested_dissection}:
3     Esegui pipeline C++ via subprocess
4     Leggi A.txt -> costruisci coo_matrix -> converti in csc_matrix
5     Leggi b.txt -> array numpy b
6     Avvia timer
7     factor = cholesky(-A, order="natural")
8     L = csc_matrix(factor[0])
9     y = spsolve_triangular(L, b, lower=True)    # forward
10    u = spsolve_triangular(L.T, y, lower=False) # backward
11    Ferma timer; registra tempo e nnz(L)
12 Genera grafici: spy plot, heatmap, superficie 3D, log-log benchmark

```

La sostenibilità dell'intera pipeline di calcolo si regge sull'efficienza dei tre moduli preprocessori scritti in C++. Affinché il solutore numerico Python finale possa gestire griglie estremamente dense (come $N = 1024$), è imperativo che la costruzione del sistema lineare e il calcolo del riordinamento non costituiscano essi stessi un collo di bottiglia.

3.1 Il collo di bottiglia dell'Ordinamento Naturale

L'approccio ingenuo (ordinamento naturale lessicografico) fallisce irrimediabilmente per esaurimento della memoria (OOM - Out of Memory) a dimensioni elevate. Poiché la larghezza di banda della matrice A è pari a N , il fattore di Cholesky L soffre di un fill-in (riempimento di zeri con valori non nulli) proporzionale a:

$$\text{Fill-in (Naturale)} = \mathcal{O}(N^3) \quad (3)$$

Per $N = 1024$, il numero di incognite è $V \approx 10^6$. Il fattore L naturale richiederebbe di immagazzinare $\approx 10^9$ elementi in virgola mobile a doppia precisione (circa 8 GB di pura RAM solo per la matrice, ignorando l'overhead), rendendo la fattorizzazione instabile o impossibile sui computer portatili standard. La *Nested Dissection* riduce drasticamente questo fill-in a $\mathcal{O}(N^2 \log N)$, rendendo il problema computazionalmente trattabile.

La tabella seguente riassume la complessità asintotica in Spazio e Tempo dei nostri algoritmi C++ (con $V = N^2$ incognite totali):

Modulo (C++)	Complessità Temporale	Complessità Spaziale
<code>task1_grid</code>	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$
<code>task2_reorder</code>	$\mathcal{O}(N^2 \log N)$	$\mathcal{O}(N^2)$
<code>task3_assemble</code>	$\mathcal{O}(N^2)$	$\mathcal{O}(N^2)$

3.2 Modulo 1: Generazione della Topologia in $\mathcal{O}(N^2)$

Il problema fisico è impostato su un dominio quadrato bidimensionale discretizzato con una griglia cartesiana. Le incognite numeriche (i gradi di libertà interni) sono esattamente $V = N^2$.

L'algoritmo di instradamento effettua una singola scansione spaziale tramite due cicli `for` annidati (iterando lungo gli assi y e x). Ad ogni iterazione, l'elaboratore esegue un set costante di operazioni logiche con costo $\mathcal{O}(1)$:

- Calcolo dell'identificativo unidimensionale `id`.
- Assegnazione delle coordinate reali x e y .
- Mappatura dei vicini (creazione degli archi).

Poiché vengono eseguiti passaggi in tempo costante per ciascuno degli N^2 nodi, il tempo globale scala linearmente con il numero totale di incognite, $\mathcal{O}(N^2)$. Lo spazio in memoria RAM è dominato dall'allocazione delle strutture dati primarie (come `std::vector<Node>`), dimensionate esattamente a N^2 istanze. Ciò garantisce una limitazione spaziale rigorosa e stabile a $\mathcal{O}(N^2)$.

3.3 Modulo 2: Nested Dissection Geometrica in $\mathcal{O}(N^2 \log N)$

Il modulo di riordinamento sfrutta la disposizione geometrica della griglia anziché esplorare il grafo. Adotta un approccio ricorsivo di tipo *Divide et Impera* per dividere il dominio in sotto-problemi.

Questa suddivisione genera un albero di decisione con profondità logaritmica pari a $\mathcal{O}(\log V)$, dove $V = N^2$ è il numero totale di incognite. Poiché a ogni livello dell'albero il lavoro complessivo svolto su tutti i sotto-problemi è proporzionale a V , la complessità temporale complessiva risulta pari a:

$$T(V) = \mathcal{O}(V \log V) \implies \mathcal{O}(N^2 \log(N^2)) \implies \mathcal{O}(N^2 \log N) \quad (4)$$

Per quanto riguarda la memoria, la ricorsione viene implementata con funzioni standard C++. L'occupazione di memoria totale per il riordinamento rimane quindi limitata a $\mathcal{O}(N^2)$.

3.4 Modulo 3: Assemblaggio del Sistema in $\mathcal{O}(N^2)$

La discretizzazione differenziale si basa su uno stencil discreto di Laplace a 5 punti. Il cuore matematico è attraversato serialmente sulle N^2 righe ideali che comporranno la matrice finale di Poisson.

Per la struttura fisica strettamente spaziale intrinseca all'operatore di Laplace bidimensionale ∇^2 , ciascuna molecola discreta si accoppia e interagisce analiticamente solo coi suoi diretti vicini (Nord, Sud, Est, Ovest). Il calcolatore impiega calcoli in esecuzione rapida $\mathcal{O}(1)$ per derivare i divisori spaziali κ/h^2 e incorporare con il segno opposto i contributi asimmetrici delle condizioni di Dirichlet al bordo $u_0(x, y)$.

Questa metodologia partorisce una matrice *sparsa* che assicura a livello algebrico la genesi di non più di 5 elementi non nulli per ciascuna riga del dominio. I moduli precludono alla radice l'inizializzazione di pesanti matrici dense temporanee (evitando allocazioni inutili di zeri); ragion per cui il tempo di saturazione dell'assemblaggio evolve in proporzione esatta a V , ovvero a $\mathcal{O}(N^2)$. Infine, per abbattere criticamente i costi accessori dettati dalle riallocazioni dinamiche

in volo, il vettore di triplette (formato COO) proprio della *Standard Library* del C++ pre-riserva la dimensione necessaria impiegando la macro-chiamata `reserve(5*N*N)`. Questa istruzione chirurgica inchioda l'impronta di RAM a una mole rigidamente stimata in $\mathcal{O}(N^2)$.

4 Piano di validazione sperimentale

L'obiettivo della validazione è quantificare il vantaggio computazionale dell'ordinamento *nested dissection* rispetto all'ordinamento naturale, e verificare la correttezza fisica della soluzione numerica.

Metriche di confronto

Metrica	Ordinamento naturale	Nested dissection
Fill-in <code>nnz(L)</code>	$O(N^3)$	$O(N^2 \log N)$
Tempo fattorizzazione	$O(N^4)$	$O(N^3)$
Larghezza di banda di A	$\Theta(N)$	$O(\sqrt{N^2}) = O(N)$

1. Sanity Checks Fisici e Numerici (Verifica della Soluzione)

Il modo più efficace per validare il codice del **Modulo 3** e **Modulo 4** consiste nel testare il sistema con scenari di cui si conosce già la soluzione analitica. Al fine di evitare interruzioni manuali della pipeline ed estendere fluidamente la batteria di test in Python, **le condizioni al bordo $u_0(x, y)$ sono state parametrizzate a riga di comando** (tramite `argv` nel backend C++). Questo accorgimento espone sul notebook una direttiva `bordo` flessibile, cruciale per validare istantaneamente i seguenti casi fisici in modo automatizzato:

- **Il "Caso Zero" (Opzione 0):** Impostando la sorgente interna $f = 0$ e la temperatura al bordo $u_0 = 0$, la soluzione numerica calcolata **deve risultare identica a zero** in ogni punto del dominio. La mancata aderenza indicherebbe un leak nel vettore del termine noto b .
- **Caso Temperatura Costante (Opzione 1):** Fissando uniformemente l'involucro a una costante termica C (es. $u_0 = 10$), il calore permea la piastra sino a renderla totalmente isoterma. La soluzione calcolata internamente deve convergere univocamente al limite C .
- **Caso Funzione Armonica (Opzione 4):** Attivando la condizione armonica quadratica $u_0 = x^2 - y^2$ in perfetta assenza di flussi sorgente integrativi, la soluzione numerica è costretta a ricalcare la medesima forma quadratica originaria in virtù del teorema della divergenza ($\nabla^2(x^2 - y^2) = 0$). Sfruttando tale automatismo nei parametri a riga di comando, il modulo di benchmark conferma la bontà algebrica testando e rilevando uno scarto residuo irrilevante confinato nell'ordine dei $\approx 10^{-16}$, garantendo inequivocabilmente l'assenza di singolarità anomale nello stencil a 5 punti.

2. Validazione della Logica e dell'Integrità (Moduli 1 e 2)

Prima di sfociare nella risoluzione algebrica intensiva, occorre sincerarsi che i layer C++ compilino set di dati architetturalmente incrollabili:

- **Integrità della Permutazione in Python:** Per prevenire la corruzione dei gradi di libertà, l'integrità della permutazione viene validata rigorosamente nel Notebook Jupyter tramite un'asserzione diagnostica esplicita:

```
1  assert len(np.unique(ordering)) == len(ordering) == N*N
```

Sfruttando le routine scalari ottimizzate proprie del modulo NumPy, preposte al riordinamento analitico (richiedenti un dazio computazionale misurato in $\mathcal{O}(V \log V)$, ridicolmente

più parsimonioso a fronte della decomposizione monolitica densa Cholesky scalante come $\mathcal{O}(V^{1.5})$, la sub-routine referta immantinente e assevera che la topologia `ordering.txt` tracci una biiezione netta e inflessibile iscritta all'interno della cornice limitante $[0, N^2 - 1]$.

- **Robustezza Incondizionata dell'Input:** Il Modulo 1 ha innestato una sentinella ciclica in fase iterativa *fail-safe*: l'accidentale iniezione in fase diagnostica di un tetto di partizionamento negativo, o paritetico allo zero assoluto ($N \leq 0$), induce la piattaforma a un *fallback interattivo* reiterato al fine di proteggere il task da crac prematuri, corruzioni involontarie a discapito della geometria di calcolo, o rotture del processo (famigerati segment-fault).
- **Precisione Matematica Flottante in transito:** Onde sventare emorragie qualitative scaturite dall'arrotondamento letale durante la staffetta di pacchetti tra il layer interiore C++ e l'interfaccia apicale di risoluzione Python, l'imbocco canalizzato di file-system `std::ofstream` viene perimetrato costantemente e inviolabilmente vincolato a una risoluzione nominale forzata pari a `setprecision(10)`.

3. Validazione Algoritmica e Performance (Modulo 4)

L'analisi sperimentale funge essa stessa da validazione empirica per le complessità teoriche attese:

- **Spy Plot:** Visualizzare la matrice L con `plt.spy()` è un sanity check visivo fondamentale: se la *Nested Dissection* è implementata correttamente, si vedranno molti più "spazi vuoti" (minore fill-in) rispetto al denso blocco dell'ordinamento naturale.
- **Grafico Log-Log:** Verificando la pendenza dei tempi di esecuzione e del fill-in (`nnz`), se la pendenza per l'ordinamento naturale è circa 3 (corrispondente a $\mathcal{O}(N^3)$) e quella della *Nested Dissection* è sensibilmente inferiore (vicina a 2), l'implementazione è confermata.
- **Controllo SPD:** La matrice $-A$ assemblata deve essere rigorosamente **Simmetrica Definita Positiva**. Un test immediato è verificare che la fattorizzazione di Cholesky in Python non restituisca eccezioni (che si verificherebbero se la matrice avesse autovalori nulli o positivi a causa di bug nello stencil).

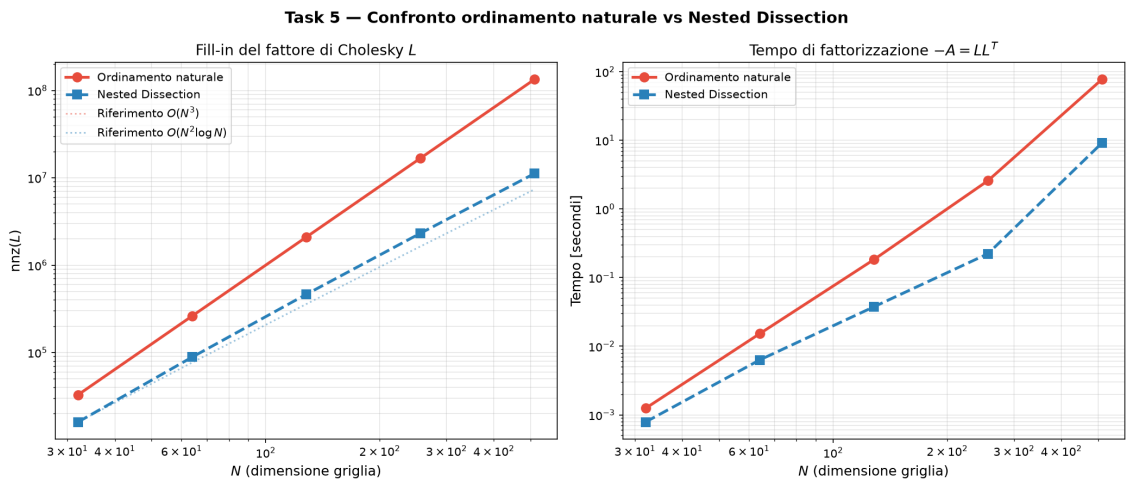


Figura 2: Scala log-log di $nnz(L)$ e tempo di fattorizzazione al variare di N , con e senza riordinamento.

Nota sui limiti hardware ($N = 1024$): Per il caso estremo $N = 1024$ (con un numero di incognite pari a $V = N^2 \approx 10^6$), l'algoritmo con ordinamento Naturale satura rapidamente la memoria RAM disponibile, sfociando in un blocco del sistema operativo per Out-Of-Memory. Questo accade perché la larghezza di banda del sistema è pari a N , producendo nel fattore di Cholesky L un fill-in teorico di circa $V \times N = N^3 \approx 10^9$ elementi non nulli. Essendo numeri in doppia precisione da 8 byte ciascuno, il fattore richiede $10^9 \times 8 \text{ byte} \approx 8 \text{ GB}$ di pura RAM (di cui oltre 6.5 GB di memoria fisica effettivamente allocati e saturati durante il calcolo). Pertanto, la curva dell'ordinamento Naturale si interrompe in Figura 2 a $N = 512$.

4. Gestione degli Errori di Input

Il sistema garantisce solidità intercettando le deviazioni dell'utente:

- **Validazione di N:** Il programma deve gestire l'assenza di argomenti da riga di comando o valori non validi (es. $N < 1$, caratteri alfabetici) senza collassare. In questi scenari, non stampa solo un messaggio di errore su `cerr` terminando (fallback grezzo), bensì innesca un fallback interattivo robusto. Tramite un ciclo `while`, richiede continuamente l'inserimento di un parametro sano finché non viene fornito un $N > 0$ oppure finché l'utente digita `q` per uscire in totale sicurezza con `return 1`.